

Practical Insights into Data Analysis and Machine Learning

PROGRAM - 8

Develop a program to implement the Naive Bayesian classifier considering Iris dataset for training. Compute the accuracy of the classifier, considering the test data.

Objective

To implement a classifier using a Naive Bayes model and evaluate its performance using accuracy on the Iris dataset.

8. Introduction

This program demonstrates the implementation of a Naive Bayes classifier using the well-known Iris dataset, which consists of measurements of iris flowers from three different species. The objective is to classify flowers based on their petal and sepal dimensions. The Naive Bayes algorithm is a simple yet powerful probabilistic classifier that applies Bayes' Theorem with the assumption of feature independence. In this program, the dataset is divided into training and testing sets, the model is trained using Gaussian Naive Bayes, and its performance is evaluated by computing the classification accuracy on the test data. This helps in understanding how Naive Bayes can be applied to solve real-world classification problems efficiently.

8.1 Bayes Theorem

Bayes theorem provides a way to calculate the probability of a hypothesis based on its prior probability, the probabilities of observing various data given the hypothesis, and the observed data itself.

Notations

- $P(h)$ prior probability of h , reflects any background knowledge about the chance that h is correct
- $P(D)$ prior probability of D , probability that D will be observed
- $P(D|h)$ probability of observing D given a world in which h holds
- $P(h|D)$ posterior probability of h , reflects confidence that h holds after D has been observed

Bayes theorem is the cornerstone of Bayesian learning methods because it provides a way to calculate the posterior probability $P(h|D)$, from the prior probability $P(h)$, together with $P(D)$ and $P(D|h)$.

Bayes Theorem:

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

- $P(h|D)$ increases with $P(h)$ and with $P(D|h)$ according to Bayes theorem.
- $P(h|D)$ decreases as $P(D)$ increases, because the more probable it is that D will be observed independent of h , the less evidence D provides in support of h .

Maximum a Posteriori (MAP) Hypothesis

- In many learning scenarios, the learner considers some set of candidate hypotheses H and is interested in finding the most probable hypothesis $h \in H$ given the observed data D . Any such maximally probable hypothesis is called a maximum a posteriori (MAP) hypothesis.
- Bayes theorem to calculate the posterior probability of each candidate hypothesis is h_{MAP} is a MAP hypothesis provided

$$\begin{aligned} h_{MAP} &= \underset{h \in H}{\operatorname{argmax}} P(h|D) \\ &= \underset{h \in H}{\operatorname{argmax}} \frac{P(D|h)P(h)}{P(D)} \\ &= \underset{h \in H}{\operatorname{argmax}} P(D|h)P(h) \end{aligned}$$

- $P(D)$ can be dropped, because it is a constant independent of h

8.2 Naïve Bayes Classifier

- The naive Bayes classifier applies to learning tasks where each instance x is described by a conjunction of attribute values and where the target function $f(x)$ can take on any value from some finite set V .
- A set of training examples of the target function is provided, and a new instance is presented, described by the tuple of attribute values $(a_1, a_2, \dots, a_m)$.
- The learner is asked to predict the target value, or classification, for this new instance.

The Bayesian approach to classifying the new instance is to assign the most probable target value, v_{MAP} , given the attribute values $(a_1, a_2, \dots, a_m)$ that describe the instance

$$v_{MAP} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j | a_1, a_2, \dots, a_n)$$

Use Bayes theorem to rewrite this expression as

$$\begin{aligned} v_{MAP} &= \underset{v_j \in V}{\operatorname{argmax}} \frac{P(a_1, a_2, \dots, a_n | v_j) P(v_j)}{P(a_1, a_2, \dots, a_n)} \\ &= \underset{v_j \in V}{\operatorname{argmax}} P(a_1, a_2, \dots, a_n | v_j) P(v_j) \quad \text{equ (1)} \end{aligned}$$

- The naive Bayes classifier is based on the assumption that the attribute values are conditionally independent given the target value. Means, the assumption is that given the target value of the instance, the probability of observing the conjunction $(a_1, a_2, \dots, a_m)$, is just the product of the probabilities for the individual attributes:

$$P(a_1, a_2, \dots, a_n | v_j) = \prod_i P(a_i | v_j)$$

Substituting this into Equation (1),

Naive Bayes classifier:

$$V_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_i P(a_i | v_j) \quad \text{equ (2)}$$

Where, V_{NB} denotes the target value output by the naive Bayes classifier

Algorithm: Naive Bayes Classifier

1. Input: Training dataset

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

Where, x_i are feature vectors and y_i are class labels.

2. Calculate the prior probability for each class:

$$P(y_k) = \frac{\text{Number of instances with class } y_k}{\text{Total number of instances}}$$

3. Compute likelihoods:

For each class y_k and for each feature x_j calculate the likelihood:

If features are continuous (like in the Iris dataset), assume Gaussian distribution:

$$P(x_j | y_k) = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}} \exp \left(-\frac{(x_j - \mu_{jk})^2}{2\sigma_{jk}^2} \right)$$

where μ_{jk} and σ_{jk} are the mean and standard deviation of feature x_j for class y_k .

4. Prediction (for a new instance x):

Compute the posterior probability for each class using Bayes' Theorem:

$$P(y_k | x) \propto P(y_k) \prod_{j=1}^n P(x_j | y_k)$$

Select the class with the highest posterior probability:

$$\hat{y} = \underset{y_k}{\operatorname{argmax}} P(y_k) \prod_{j=1}^n P(x_j | y_k)$$

5. Output: Predicted $\hat{y}$ for the new instance.

8.3 Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = load_iris()
X = iris.data # Features
y = iris.target # Labels

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

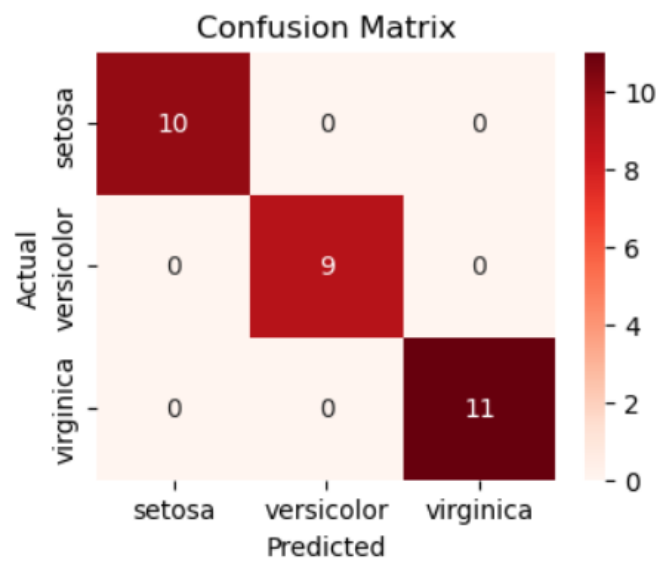
# Initialize the Gaussian Naive Bayes classifier
model = GaussianNB()

# Train the classifier on the training data
model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = model.predict(X_test)

# Calculate the accuracy of the classifier
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of the Naive Bayes classifier: {accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# The confusion matrix to visualize the performance
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(4, 3))
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Viva Questions

- What is Bayes' Theorem? State its mathematical formula.
- What is the meaning of "naive" in Naive Bayes?
- Why is the Naive Bayes classifier called a probabilistic classifier?
- What assumptions does the Naive Bayes classifier make about the features?
- What is the role of the prior and likelihood in Bayes' Theorem?
- What is the Maximum a Posteriori (MAP) estimate? How is it used in classification?
- What is the difference between prior probability and posterior probability?
- Why do we assume Gaussian distribution in Naive Bayes for continuous features?
- How does Naive Bayes handle categorical vs. continuous data?
- Why can we ignore $P(D)$ (evidence) in MAP estimation?
- What are the advantages and limitations of the Naive Bayes algorithm?